Python Built-in Functions

Complete Reference — All 68 Functions · Detailed Explanations · Multiple Examples · Real-World Uses

This comprehensive guide covers every built-in function in Python 3 with full explanations, multiple code examples showing different use cases, expected output, real-world applications, practical tips, and important warnings. Perfect for beginners and as a quick reference for experienced developers.

Category	Functions
Math & Numbers	abs, bin, complex, divmod, float, hex, int, oct, pow, round
Sequences & Collections	all, any, dict, enumerate, filter, frozenset, list, map, max, min, range, reversed, set, sorted, sum, tuple, zip
Strings & Characters	ascii, bytes, bytearray, chr, format, ord, repr, str
Object & Class	callable, classmethod, delattr, dir, getattr, hasattr, id, isinstance, issubclass, object, property, setattr, slice, staticmethod, super, type, vars
I/O & Runtime	breakpoint, compile, eval, exec, globals, hash, help, input, iter, locals, memoryview, next, open, print, __import__

A

abs()

Syntax: abs(x)

Returns the absolute value of a number — i.e. its distance from zero, always non-negative. Works with integers, floats, and complex numbers (returns magnitude for complex).

■ **Tip:** Use abs() when you only care about magnitude, not direction — e.g. tolerances, errors, distances.

■ **Basic integers & floats**

```
print(abs(-10))      # positive integer
print(abs(10))       # already positive
print(abs(-3.75))    # float
print(abs(0))        # zero
```

Output:

```
10
10
3.75
0
```

■ **Complex numbers**

```
print(abs(3 + 4j))   # sqrt(3^2 + 4^2) = 5.0
print(abs(-2 + 0j))  # magnitude of complex
```

Output:

```
5.0
2.0
```

■ **Real-world: distance & error calculation**

```
predicted = 95
actual    = 87
error = abs(predicted - actual)
print(f'Prediction error: {error}')  # always positive

# Clamp a value within range
def clamp(val, lo, hi):
    if abs(val - lo) != val - lo:  # val < lo
        return lo
    return min(val, hi)
```

Output:

```
Prediction error: 8
```

all()

Syntax: all(iterable)

Returns True if every element in the iterable is truthy, OR if the iterable is empty. Returns False as soon as any element is falsy (short-circuits for efficiency).

■ **Note:** all([]) returns True (vacuous truth). Be careful with empty inputs.

■ **Tip:** all() with a generator expression is memory-efficient — it stops checking the moment it finds a False.

■ **Basic examples**

```
print(all([1, 2, 3]))          # all truthy → True
print(all([1, 0, 3]))          # 0 is falsy → False
print(all([]))                 # empty → True
print(all(['a','b','']))       # '' is falsy → False
```

Output:

```
True
False
True
False
```

■ **With conditions using generator**

```
nums = [4, 8, 12, 16]
print(all(n % 4 == 0 for n in nums))  # all divisible by 4?

words = ['hello','world','python']
print(all(len(w) >= 5 for w in words))  # all long words?
```

Output:

```
True
True
```

■ **Real-world: form validation**

```
fields = {'name': 'Alice', 'email': 'a@b.com', 'age': '25'}
is_valid = all(v.strip() != '' for v in fields.values())
print('Form OK:', is_valid)

# Check all students passed (score >= 50)
scores = [78, 92, 55, 61, 88]
print('All passed:', all(s >= 50 for s in scores))
```

Output:

```
Form OK: True
All passed: True
```

any()

```
Syntax: any(iterable)
```

Returns True if at least one element in the iterable is truthy. Returns False if the iterable is empty or all elements are falsy. Short-circuits on the first truthy element.

■ **Tip:** any() vs all(): any() = OR logic, all() = AND logic. Use together for powerful validations.

■ Basic examples

```
print(any([0, 0, 1]))     # one truthy → True
print(any([0, 0, 0]))     # all falsy  → False
print(any([]))            # empty      → False
print(any([None,'','x'])) # 'x' is truthy → True
```

Output:

```
True
False
False
True
```

■ Searching in data

```
names = ['Alice','Bob','Charlie']
print(any('Charlie' == n for n in names))  # found?

prices = [200, 450, 150, 899]
print(any(p > 500 for p in prices))  # any expensive?
```

Output:

```
True
True
```

■ Real-world: permission check

```
user_roles = ['viewer', 'editor']
admin_roles = ['admin', 'superuser', 'editor']
has_access = any(r in admin_roles for r in user_roles)
print('Access granted:', has_access)
```

Output:

```
Access granted: True
```

ascii()

Syntax: ascii(object)

Returns a string like repr() but escapes all non-ASCII characters using \x, \u, or \U escape sequences. Useful when you need a plain ASCII-safe representation.

■ **Tip:** Use ascii() when writing to systems that only handle ASCII — like certain log files or old APIs.

■ **Basic usage**

```
print(ascii('hello'))        # plain ASCII → no change
print(ascii('caf\u00e9'))   # é escaped
print(ascii('■■■'))       # Japanese escaped
print(ascii(['a','\u03b1'])) # alpha symbol
```

Output:

```
'hello'
'caf\xe9'
'\u65e5\u672c\u8a9e'
['a', '\u03b1']
```

■ **Practical: safe logging**

```
user_input = 'Ren\u00e9'    # René
safe_log = ascii(user_input)
print('Logging:', safe_log)  # safe for ASCII-only logs
```

Output:

```
Logging: 'Ren\xe9'
```

B

bin()

Syntax: bin(x)

Converts an integer to a binary string prefixed with '0b'. The input must be an integer (or an object implementing __index__). Useful for bitwise operations and low-level programming.

■ **Tip:** For padding, use f'{n:08b}' — gives 8-bit binary with zero padding.

■ Basic conversions

```
print(bin(0))    # → '0b0'
print(bin(1))    # → '0b1'
print(bin(10))   # → '0b1010'
print(bin(255))  # → '0b11111111'
print(bin(-5))   # → '-0b101'
```

Output:

```
0b0
0b1
0b1010
0b11111111
-0b101
```

■ Formatting without prefix

```
n = 42
print(bin(n)[2:])          # strip '0b'
print(format(n, 'b'))      # same, cleaner
print(f'{n:08b}')          # zero-padded 8 bits
```

Output:

```
101010
101010
00101010
```

■ Real-world: check if power of 2

```
def is_power_of_two(n):
    return n > 0 and bin(n).count('1') == 1

print(is_power_of_two(16))  # True
print(is_power_of_two(18))  # False
```

Output:

```
True
False
```

bool()

Syntax: bool(x=False)

Converts a value to Boolean True or False. Python's truth table: 0, None, '', [], {}, set(), () all evaluate to False. Everything else is True.

Tip: You rarely need to call bool() explicitly — Python evaluates truthiness automatically in if/while.

Falsy values

```
print(bool(0))       # int zero
print(bool(0.0))     # float zero
print(bool(''))      # empty string
print(bool([]))      # empty list
print(bool(None))    # None
print(bool({}))      # empty dict
```

Output:

```
False
False
False
False
False
False
```

Truthy values

```
print(bool(1))       # any non-zero int
print(bool(-5))      # negative also truthy
print(bool('hi'))    # non-empty string
print(bool([0]))     # list with one element
print(bool({'a':1})) # non-empty dict
```

Output:

```
True
True
True
True
True
```

Real-world: clean input handling

```
def has_value(x):
    return bool(x)  # clean way to check

name = input_from_user = ''
if not has_value(name):
    print('Name is required!')

# Convert 0/1 from database to bool
db_value = 1
is_active = bool(db_value)
print('User active:', is_active)
```

Output:

```
Name is required!
User active: True
```

bytearray()

Syntax: bytearray(source, encoding, errors)

Returns a mutable sequence of bytes. Each element is an integer 0–255. Unlike bytes, bytearray can be modified in-place. Great for binary data manipulation, networking, and file I/O.

■ **Tip:** Use bytearray when you need to modify binary data. Use bytes for immutable binary data.

■ Creating bytearrays

```
ba1 = bytearray(5)             # 5 zero bytes
ba2 = bytearray(b'hello')      # from bytes literal
ba3 = bytearray('hi', 'utf-8') # from string
ba4 = bytearray([72,101,108])  # from int list
print(ba1)  # bytearray(b'\x00\x00\x00\x00\x00')
print(ba2)  # bytearray(b'hello')
print(ba4)  # bytearray(b'Hel')
```

Output:

```
bytearray(b'\x00\x00\x00\x00\x00')
bytearray(b'hello')
bytearray(b'Hel')
```

■ Modifying in-place

```
ba = bytearray(b'hello')
ba[0] = 72          # change 'h' (104) to 'H' (72)
ba.append(33)       # append '!'
print(ba.decode())  # → 'Hello!'
```

Output:

```
Hello!
```

■ Real-world: XOR encryption

```
def xor_encrypt(data: str, key: int) -> bytearray:
    ba = bytearray(data.encode())
    for i in range(len(ba)):
        ba[i] ^= key
    return ba

enc = xor_encrypt('Secret', 42)
dec = xor_encrypt(enc.decode('latin-1'), 42)
print('Decrypted:', dec.decode())
```

Output:

```
Decrypted: Secret
```

bytes()

Syntax: bytes(source, encoding, errors)

Returns an immutable sequence of bytes (integers 0–255). Used for binary data, network protocols, file I/O, and encoding text to raw bytes.

Tip: bytes is immutable — use bytearray if you need to change individual byte values.

Creating bytes objects

```
b1 = bytes(4)               # 4 zero bytes
b2 = bytes([65, 66, 67])    # from list → b'ABC'
b3 = bytes('hello','utf-8') # encode string
b4 = b'world'               # literal syntax
print(b2)   # b'ABC'
print(b3)   # b'hello'
```

Output:

```
b'ABC'
b'hello'
```

Encoding and decoding

```
text = 'Python'
encoded = bytes(text, 'utf-8')
print(encoded)              # b'Python'
print(encoded.decode())     # back to 'Python'
print(list(encoded))        # [80,121,116,104,111,110]
```

Output:

```
b'Python'
Python
[80, 121, 116, 104, 111, 110]
```

Real-world: reading binary file

```
# Reading a file as raw bytes
with open('image.png','rb') as f:
    header = f.read(8)      # first 8 bytes
    is_png = header[:4] == bytes([137,80,78,71])
    print('Is PNG:', is_png)
```

Output:

```
Is PNG: True
```

C

callable()

Syntax: callable(object)

Returns True if the object appears callable (can be called like a function). Classes, functions, methods, and objects with __call__ return True. Integers, strings, etc. return False.

■ **Tip:** callable() is useful for building flexible APIs that accept either a value or a function.

■ **Basic checks**

```
print(callable(print))        # built-in function
print(callable(len))          # built-in
print(callable(42))           # int → False
print(callable('hello'))      # string → False
print(callable(list))         # class → True
```

Output:

```
True
True
False
False
True
```

■ **Custom callable class**

```
class Multiplier:
    def __init__(self, factor):
        self.factor = factor
    def __call__(self, x):
        return x * self.factor

double = Multiplier(2)
print(callable(double))    # True — has __call__
print(double(5))           # 10
```

Output:

```
True
10
```

■ **Real-world: safe function dispatch**

```
def run_if_callable(obj, *args):
    if callable(obj):
        return obj(*args)
    return obj   # return value as-is

print(run_if_callable(lambda x: x*2, 5))  # 10
print(run_if_callable(99))                # 99
```

Output:

```
10
99
```

chr()

Syntax: chr(i)

Returns the string of one character whose Unicode code point is the integer i. The inverse of ord(). Valid range is 0 to 1,114,111 (0x10FFFF). Raises ValueError outside this range.

■ **Tip:** chr() and ord() are inverses: chr(ord('A')) == 'A' and ord(chr(65)) == 65.

■ Basic usage

```
print(chr(65))    # uppercase A
print(chr(97))    # lowercase a
print(chr(48))    # digit 0
print(chr(9829))  # heart symbol ♥
print(chr(128512))# ■ emoji
```

Output:

```
A
a
0
♥
■
```

■ Generate alphabet

```
lower = [chr(i) for i in range(97, 123)]
print(''.join(lower))

upper = [chr(i) for i in range(65, 91)]
print(''.join(upper))
```

Output:

```
abcdefghijklmnopqrstuvwxyz
ABCDEFGHIJKLMNOPQRSTUVWXYZ
```

■ Real-world: Caesar cipher

```
def caesar(text, shift):
    result = []
    for c in text:
        if c.isalpha():
            base = 65 if c.isupper() else 97
            result.append(chr((ord(c)-base+shift)%26+base))
        else:
            result.append(c)
    return ''.join(result)

print(caesar('Hello World', 3))   # Khoor Zruog
print(caesar('Khoor Zruog', -3))  # Hello World
```

Output:

```
Khoor Zruog
Hello World
```

complex()

Syntax: complex(real=0, imag=0)

Creates a complex number with a real and imaginary part. Can also parse a string like '3+4j'. Complex numbers are written as (real+imagj) in Python.

Tip: Python uses 'j' not 'i' for imaginary unit: write 3+4j directly without calling complex().

Creating complex numbers

```
print(complex(3, 4))     # 3+4j
print(complex(5))        # 5+0j
print(complex(0, -1))    # -1j
print(complex('2+3j'))   # parse string
print(complex('-1j'))    # pure imaginary
```

Output:

```
(3+4j)
(5+0j)
-1j
(2+3j)
-1j
```

Properties and operations

```
c = complex(3, 4)
print(c.real)         # 3.0
print(c.imag)         # 4.0
print(abs(c))         # magnitude = 5.0
print(c.conjugate())  # (3-4j)
print(c * complex(1,-1))  # multiply
```

Output:

```
3.0
4.0
5.0
(3-4j)
(7+1j)
```

D

dict()

Syntax: dict(**kwargs) / dict(mapping) / dict(iterable)

Creates a new dictionary. Can be built from keyword arguments, another mapping, or an iterable of key-value pairs. Dictionaries store key→value pairs and preserve insertion order (Python 3.7+).

■ **Tip:** Use dict.get(key, default) instead of dict[key] to avoid KeyError on missing keys.

■ Different ways to create

```
d1 = dict(name='Alice', age=30)      # keyword args
d2 = dict([('x',1),('y',2)])         # from pairs
d3 = dict({'a':1,'b':2})             # from dict
d4 = dict(zip(['a','b'],[1,2]))      # from zip
print(d1)  # {'name': 'Alice', 'age': 30}
print(d4)  # {'a': 1, 'b': 2}
```

Output:

```
{'name': 'Alice', 'age': 30}
{'a': 1, 'b': 2}
```

■ Common operations

```
d = dict(a=1, b=2, c=3)
print(d.get('a'))       # 1
print(d.get('z', 0))    # default 0
d['d'] = 4              # add key
del d['a']              # remove key
print(list(d.keys()))   # ['b','c','d']
print(list(d.values())) # [2, 3, 4]
```

Output:

```
1
0
['b', 'c', 'd']
[2, 3, 4]
```

■ Real-world: word frequency counter

```
text = 'the cat sat on the mat the cat'
freq = dict()
for word in text.split():
    freq[word] = freq.get(word, 0) + 1
print(sorted(freq.items(), key=lambda x:-x[1]))
```

Output:

```
[('the',3),('cat',2),('sat',1),('on',1),('mat',1)]
```

dir()

Syntax: dir(object=None)

Returns a list of names in the current scope (no argument), or a list of attributes and methods of an object. Very useful for exploration and debugging.

Tip: Use dir() in the Python shell to quickly discover what methods/attributes an object has.

Exploring objects

```
print(dir([]))       # list methods
# ['append','clear','copy','count','extend',
# 'index','insert','pop','remove','reverse','sort']

print(dir(str))      # string methods — very long list
print(dir({}))       # dict methods
```

Output:

```
[...list methods...]
```

Filter non-dunder attributes

```
methods = [m for m in dir([]) if not m.startswith('_')]
print(methods)
```

Output:

```
['append','clear','copy','count','extend','index','insert','pop','remove','reverse','sort']
```

Explore custom class

```
class Dog:
    breed = 'Labrador'
    def bark(self): pass
    def fetch(self): pass

own = [a for a in dir(Dog) if not a.startswith('_')]
print(own)   # ['bark', 'breed', 'fetch']
```

Output:

```
['bark', 'breed', 'fetch']
```

divmod()

Syntax: divmod(a, b)

Returns a tuple (quotient, remainder) — equivalent to (a // b, a % b) but computed in one step. Works with integers and floats.

■ **Tip:** divmod() is faster than computing // and % separately — Python does it in a single operation.

■ **Basic integer division**

```
print(divmod(17, 5))   # 17 = 5*3 + 2
print(divmod(100, 7))  # 100 = 7*14 + 2
print(divmod(10, 10))  # exact division
print(divmod(0, 5))    # zero numerator
```

Output:

```
(3, 2)
(14, 2)
(1, 0)
(0, 0)
```

■ **Float division**

```
print(divmod(7.5, 2.5))   # float works too
print(divmod(-7, 2))      # negative numbers
```

Output:

```
(3.0, 0.0)
(-4, 1)
```

■ **Real-world: time conversion**

```
def seconds_to_hms(total_seconds):
    mins, secs = divmod(total_seconds, 60)
    hours, mins = divmod(mins, 60)
    return f'{hours:02d}:{mins:02d}:{secs:02d}'

print(seconds_to_hms(3661))   # 1 hr 1 min 1 sec
print(seconds_to_hms(86399))  # 23:59:59
```

Output:

```
01:01:01
23:59:59
```

E

enumerate()

Syntax: enumerate(iterable, start=0)

Adds a counter to an iterable and returns an enumerate object of (index, value) pairs. The start parameter sets the beginning index. Eliminates the need for manual counter variables.

■ **Tip:** Always prefer enumerate() over range(len(list)) — it's more Pythonic and readable.

■ **Basic usage**

```
fruits = ['apple','banana','cherry']
for i, fruit in enumerate(fruits):
    print(i, fruit)
```

Output:

```
0 apple
1 banana
2 cherry
```

■ **Custom start index**

```
for i, fruit in enumerate(fruits, start=1):
    print(f'{i}. {fruit}')
```

Output:

```
1. apple
2. banana
3. cherry
```

■ **Real-world: find index of item & update**

```
scores = [80, 92, 71, 88, 60]
for i, score in enumerate(scores):
    if score < 70:
        scores[i] = 70   # minimum score policy
print(scores)  # 60 → 70

# Build a numbered menu
menu = ['New Game','Load','Settings','Quit']
for n, item in enumerate(menu, 1):
    print(f'  [{n}] {item}')
```

Output:

```
[80, 92, 71, 88, 70]
[1] New Game
[2] Load
[3] Settings
[4] Quit
```

eval()

Syntax: eval(expression, globals=None, locals=None)

Parses and evaluates a Python expression string and returns the result. Can optionally take custom globals/locals dictionaries. Only evaluates expressions, not statements.

■ **Note:** Avoid eval() in production for user input. Use ast.literal_eval() for safe parsing of literals.

■ **Tip:** eval() can execute arbitrary code — never use it on untrusted user input without sanitization.

■ **Basic evaluation**

```
print(eval('2 + 3'))           # math
print(eval('10 * 5 - 8'))      # expression
print(eval('[1,2,3]'))         # list literal
print(eval('"hello".upper()'))# method call
```

Output:

```
5
42
[1, 2, 3]
HELLO
```

■ **With variables**

```
x = 10
print(eval('x * 2'))           # uses local x
print(eval('x + y', {'x':5,'y':3}))  # custom vars
```

Output:

```
20
8
```

■ **Real-world: simple calculator**

```
def safe_calc(expr):
    allowed = {'+','-','*','/','(',')','.',' '}
    if all(c.isdigit() or c in allowed for c in expr):
        return eval(expr)
    return 'Invalid expression'

print(safe_calc('(3 + 4) * 2'))  # 14
print(safe_calc('2 ** 8'))       # Invalid
```

Output:

```
14
Invalid expression
```

F

filter()

Syntax: filter(function, iterable)

Constructs an iterator from elements of an iterable for which the function returns True. If function is None, removes all falsy values. Returns a filter object (use list() to see results).

Tip: List comprehensions often replace filter(): [x for x in nums if x%2==0] is equally readable.

Basic filtering

```
nums = [1,2,3,4,5,6,7,8,9,10]
evens = list(filter(lambda x: x%2==0, nums))
print(evens)   # [2,4,6,8,10]

odds = list(filter(lambda x: x%2!=0, nums))
print(odds)    # [1,3,5,7,9]
```

Output:

```
[2, 4, 6, 8, 10]
[1, 3, 5, 7, 9]
```

Remove falsy values (None as function)

```
data = [1, None, 2, '', 3, 0, 4, False]
clean = list(filter(None, data))
print(clean)   # [1, 2, 3, 4]
```

Output:

```
[1, 2, 3, 4]
```

Real-world: filter students who passed

```
students = [
    {'name':'Alice','score':85},
    {'name':'Bob',  'score':42},
    {'name':'Carol','score':91},
    {'name':'Dave', 'score':58},
]
passed = list(filter(lambda s: s['score']>=50, students))
print([s['name'] for s in passed])
```

Output:

```
['Alice', 'Carol', 'Dave']
```

float()

Syntax: `float(x=0.0)`

Converts a number or string to a floating-point number. Accepts integers, strings like '3.14' or '1e-5', and the special values 'inf', '-inf', 'nan'.

■ **Tip:** Use try/except ValueError when converting user input — strings like 'hello' will raise an error.

■ Basic conversion

```
print(float(7))          # int → float
print(float('3.14'))     # string → float
print(float('1e-5'))     # scientific notation
print(float('inf'))      # infinity
print(float('-inf'))     # negative infinity
print(float('nan'))      # not a number
```

Output:

```
7.0
3.14
1e-05
inf
-inf
nan
```

■ Arithmetic with floats

```
x = float('3.14159')
print(round(x, 2))   # 3.14
print(x > 3)         # True
print(int(x))        # 3 (truncates)
```

Output:

```
3.14
True
3
```

■ Real-world: parse sensor data

```
raw_readings = ['23.5','24.1','error','22.8','25.0']
valid = []
for r in raw_readings:
    try:
        valid.append(float(r))
    except ValueError:
        print(f'Skipping bad reading: {r}')
print('Avg:', sum(valid)/len(valid))
```

Output:

```
Skipping bad reading: error
Avg: 23.85
```

format()

```
Syntax: format(value, format_spec='')
```

Formats a value using a format specification. Calls the value's __format__ method. Equivalent to the format mini-language used in f-strings and str.format().

> ■ **Tip:** In f-strings, use the same spec after a colon: f'{3.14:.2f}' is the same as format(3.14,'.2f').

■ Numbers

```
print(format(3.14159, '.2f'))   # 2 decimal places
print(format(1000000, ','))     # thousands separator
print(format(0.25, '%'))        # percentage
print(format(255, 'x'))         # hexadecimal
print(format(255, 'b'))         # binary
print(format(255, 'o'))         # octal
```

Output:

```
3.14
1,000,000
25.000000%
ff
11111111
377
```

■ Padding and alignment

```
print(format('hi',  '<10'))  # left align  in 10
print(format('hi',  '>10'))  # right align in 10
print(format('hi',  '^10'))  # center      in 10
print(format(42,   '05d'))  # zero-pad to width 5
```

Output:

```
'hi        '
'        hi'
'    hi    '
'00042'
```

■ Real-world: pretty table printing

```
products = [('Apple',1.5,100),('Banana',0.3,250)]
print(f'{'Item':<12} {'Price':>8} {'Qty':>6}')
for name,price,qty in products:
    print(f'{name:<12} {format(price,">7.2f")} {qty:>6}')
```

Output:

```
Item          Price    Qty
Apple           1.50    100
Banana          0.30    250
```

frozenset()

Syntax: frozenset(iterable=())

Creates an immutable set. Like set() but cannot be modified after creation. Because it's immutable and hashable, frozensets can be used as dictionary keys or elements of another set.

■ **Tip:** Use frozenset when you need a set that won't change — e.g. for caching or as a dict key.

■ **Creating and using frozensets**

```
fs = frozenset([1, 2, 3, 2, 1])  # duplicates removed
print(fs)              # frozenset({1, 2, 3})
print(type(fs))        # <class 'frozenset'>
# fs.add(4)            # AttributeError — immutable!
```

Output:

```
frozenset({1, 2, 3})
```

■ **As dict keys (sets can't do this)**

```
permissions = {
    frozenset(['read','write']): 'Editor',
    frozenset(['read']):         'Viewer',
}
user_perms = frozenset(['read','write'])
print(permissions[user_perms])   # Editor
```

Output:

```
Editor
```

■ **Set operations**

```
a = frozenset([1,2,3,4])
b = frozenset([3,4,5,6])
print(a & b)   # intersection: {3, 4}
print(a | b)   # union: {1,2,3,4,5,6}
print(a - b)   # difference: {1, 2}
```

Output:

```
frozenset({3, 4})
frozenset({1,2,3,4,5,6})
frozenset({1, 2})
```

G

getattr()

Syntax: `getattr(object, name, default=None)`

Gets the value of the named attribute of an object. If the attribute doesn't exist, returns the default (if provided) or raises AttributeError. Useful for dynamic attribute access.

Tip: getattr() is the foundation of Python's dynamic dispatch — many frameworks use it internally.

Basic usage

```
class Person:
    name = 'Alice'
    age  = 30

p = Person()
print(getattr(p,'name'))           # 'Alice'
print(getattr(p,'age'))            # 30
print(getattr(p,'email','N/A'))    # default 'N/A'
```

Output:

```
Alice
30
N/A
```

Dynamic method calling

```
class Calc:
    def add(self,a,b): return a+b
    def mul(self,a,b): return a*b

c = Calc()
op = 'add'
method = getattr(c, op)   # get method by name
print(method(3, 4))       # 7
```

Output:

```
7
```

Real-world: config object access

```
class Config:
    debug = True
    db_host = 'localhost'
    db_port = 5432

cfg = Config()
keys = ['debug','db_host','db_port','timeout']
for k in keys:
    print(f'{k}: {getattr(cfg,k,"not set")}' )
```

Output:

```
debug: True
db_host: localhost
db_port: 5432
timeout: not set
```

H

hash()

Syntax: hash(object)

Returns the hash value of an object as an integer. Used internally by dict and set for fast lookups. Only immutable (hashable) objects can be hashed.

■ **Note:** Mutable objects (list, dict, set) are not hashable and will raise TypeError.

■ **Tip:** Hash values can differ across Python sessions (hash randomization). Don't persist hash values.

■ **Basic hashing**

```
print(hash(42))          # integer
print(hash(3.14))        # float
print(hash('hello'))     # string
print(hash((1,2,3)))     # tuple (immutable)
# hash([1,2,3])          # TypeError — list is mutable
```

Output:

```
42
some_int
some_int
some_int
```

■ **Consistent for equal objects**

```
print(hash(1) == hash(1.0))   # True — equal objects same hash
print(hash('abc') == hash('abc'))  # True
```

Output:

```
True
True
```

■ **Real-world: simple cache**

```
cache = {}
def expensive(n):
    key = hash(n)
    if key not in cache:
        cache[key] = n * n  # simulate expensive op
    return cache[key]

print(expensive(10))  # computed
print(expensive(10))  # from cache
```

Output:

```
100
100
```

hex()

```
Syntax: hex(x)
```

Converts an integer to a lowercase hexadecimal string prefixed with '0x'. The input must be an integer. Negative numbers include a minus sign.

> ■ **Tip:** For CSS/HTML colors use f'#{r:02x}{g:02x}{b:02x}' — it's cleaner than calling hex() three times.

■ Basic conversions

```
print(hex(0))     # 0x0
print(hex(255))   # 0xff
print(hex(256))   # 0x100
print(hex(-42))   # -0x2a
print(hex(16))    # 0x10
```

Output:

```
0x0
0xff
0x100
-0x2a
0x10
```

■ Formatting hex without prefix

```
n = 255
print(hex(n)[2:])         # 'ff'
print(format(n, 'X'))     # 'FF' (uppercase)
print(format(n, '04x'))   # '00ff' (padded)
print(f'#{n:06x}')        # '#0000ff' CSS color
```

Output:

```
ff
FF
00ff
#0000ff
```

■ Real-world: RGB to hex color

```
def rgb_to_hex(r, g, b):
    return f'#{r:02x}{g:02x}{b:02x}'

print(rgb_to_hex(255, 99, 71))   # tomato red
print(rgb_to_hex(0, 128, 0))     # green
print(rgb_to_hex(30, 144, 255))  # dodger blue
```

Output:

```
#ff6347
#008000
#1e90ff
```

I

id()

Syntax: id(object)

Returns the unique integer identity of an object — its memory address in CPython. Two objects with non-overlapping lifetimes may have the same id. The 'is' operator compares ids.

■ **Note:** Never use 'is' to compare values like strings or numbers — use == instead.

■ **Tip:** Use 'is' (which compares id) to check if two variables point to the SAME object, not equal value.

■ Basic id usage

```
x = [1, 2, 3]
y = x            # same object
z = [1, 2, 3]    # different object, same value
print(id(x) == id(y))   # True — same object
print(id(x) == id(z))   # False — different object
print(x is y)            # True
print(x is z)            # False
```

Output:

```
True
False
True
False
```

■ Integer interning (CPython)

```
a = 256
b = 256
print(a is b)   # True — small ints are cached

c = 1000
d = 1000
print(c is d)   # may be False — large ints not cached
```

Output:

```
True
False
```

input()

Syntax: `input(prompt='')`

Reads a line of text from the user via stdin. The optional prompt string is displayed first. Always returns a string — convert with int(), float() etc. as needed.

■ **Tip:** Always wrap input() in try/except when converting to int/float to handle invalid user input.

■ Basic input

```
name = input('Enter your name: ')
print(f'Hello, {name}!')
```

Output:

```
Enter your name: Alice
Hello, Alice!
```

■ Converting input types

```
age = int(input('Your age: '))    # convert to int
gpa = float(input('Your GPA: '))  # convert to float
print(f'In 10 years: {age+10}')
```

Output:

```
Your age: 20
Your GPA: 3.8
In 10 years: 30
```

■ Real-world: input validation loop

```
while True:
    try:
        n = int(input('Enter a positive number: '))
        if n > 0:
            break
        print('Must be positive!')
    except ValueError:
        print('Numbers only please!')
print(f'You entered: {n}')
```

Output:

```
Enter a positive number: -1
Must be positive!
Enter a positive number: abc
Numbers only please!
Enter a positive number: 5
You entered: 5
```

int()

Syntax: int(x=0, base=10)

Converts a number or string to an integer. Truncates floats towards zero. With base parameter, parses strings in different number bases (2=binary, 8=octal, 16=hex).

■ **Tip:** int() truncates floats — use round() first if you want rounding: int(round(3.7)) → 4.

■ Basic conversion

```
print(int(3.9))        # 3 — truncates, not rounds
print(int(-3.9))       # -3 — towards zero
print(int('42'))       # string to int
print(int(True))       # bool → 1
print(int(False))      # bool → 0
```

Output:

```
3
-3
42
1
0
```

■ Different bases

```
print(int('1010', 2))   # binary → 10
print(int('ff',  16))   # hex    → 255
print(int('77',   8))   # octal  → 63
print(int('0xff',16))   # with prefix → 255
```

Output:

```
10
255
63
255
```

■ Real-world: parse hex color

```
def hex_to_rgb(hex_color):
    h = hex_color.lstrip('#')
    r = int(h[0:2], 16)
    g = int(h[2:4], 16)
    b = int(h[4:6], 16)
    return (r, g, b)

print(hex_to_rgb('#ff6347'))  # (255, 99, 71)
print(hex_to_rgb('#1e90ff'))  # (30, 144, 255)
```

Output:

```
(255, 99, 71)
(30, 144, 255)
```

isinstance()

Syntax: isinstance(object, classinfo)

Returns True if the object is an instance of the class or a subclass thereof. classinfo can be a single class or a tuple of classes. Much preferred over type() for type checking.

■ **Tip:** Prefer isinstance() over type(x) == int because isinstance() respects class inheritance.

■ **Basic checks**

```
print(isinstance(42, int))          # True
print(isinstance(3.14, float))      # True
print(isinstance('hi', str))        # True
print(isinstance(True, int))        # True — bool is subclass of int
print(isinstance([1,2], list))      # True
```

Output:

```
True
True
True
True
True
```

■ **Multiple types at once**

```
def is_number(x):
    return isinstance(x, (int, float, complex))

print(is_number(5))      # True
print(is_number(3.14))   # True
print(is_number('5'))    # False
```

Output:

```
True
True
False
```

■ **Real-world: flexible function**

```
def process(data):
    if isinstance(data, str):
        return data.upper()
    elif isinstance(data, list):
        return [process(x) for x in data]
    elif isinstance(data, (int,float)):
        return data * 2
    return data

print(process('hello'))       # HELLO
print(process([1,'hi',2]))    # [2,'HI',4]
```

Output:

```
HELLO
[2, 'HI', 4]
```

L

len()

Syntax: `len(s)`

Returns the number of items (length) in a container. Works with strings, lists, tuples, dicts, sets, ranges, and any object implementing __len__. Runs in O(1) — instant regardless of size.

■ **Tip:** len() is O(1) for built-in types — Python stores the length, it doesn't count every time.

■ **All container types**

```
print(len('hello'))        # 5 — string chars
print(len([1,2,3,4]))      # 4 — list items
print(len((1,2)))          # 2 — tuple items
print(len({1,2,3}))        # 3 — set items
print(len({'a':1,'b':2}))  # 2 — dict keys
print(len(range(100)))     # 100 — range
```

Output:

```
5
4
2
3
2
100
```

■ **Guard against empty**

```
data = []
if len(data) == 0:
    print('Empty!')
# Pythonic version:
if not data:
    print('Empty!')
```

Output:

```
Empty!
Empty!
```

■ **Real-world: pagination**

```
def paginate(items, page_size=10):
    total = len(items)
    pages = (total + page_size - 1) // page_size
    print(f'{total} items across {pages} pages')
    return [items[i:i+page_size]
            for i in range(0, total, page_size)]

data = list(range(25))
pages = paginate(data, 10)
print(f'Page 1: {pages[0]}')
```

Output:

```
25 items across 3 pages
Page 1: [0,1,2,3,4,5,6,7,8,9]
```

list()

Syntax: `list(iterable=())`

Creates a mutable list from an iterable, or an empty list if no argument. Lists are ordered, allow duplicates, and support indexing/slicing.

■ **Tip:** list() creates a shallow copy when passed another list — changes to nested objects still affect both.

■ Creating lists

```
print(list())             # [] — empty
print(list('abc'))        # ['a','b','c']
print(list(range(5)))     # [0,1,2,3,4]
print(list((1,2,3)))      # from tuple
print(list({3,1,2}))      # from set (unordered!)
```

Output:

```
[]
['a','b','c']
[0,1,2,3,4]
[1,2,3]
[1,2,3]
```

■ List operations

```
lst = list(range(5))   # [0,1,2,3,4]
lst.append(5)          # [0,1,2,3,4,5]
lst.insert(0, -1)      # [-1,0,1,2,3,4,5]
lst.pop()              # removes 5
lst.sort(reverse=True) # [4,3,2,1,0,-1]
print(lst)
```

Output:

```
[4, 3, 2, 1, 0, -1]
```

■ Real-world: convert generator to list

```
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        yield a
        a, b = b, a+b

fibs = list(fibonacci(10))
print(fibs)
```

Output:

```
[0,1,1,2,3,5,8,13,21,34]
```

M

map()

Syntax: map(function, iterable, ...)

Applies a function to every item in one or more iterables and returns a map iterator. With multiple iterables, applies the function with corresponding elements from each (like zip).

Tip: list comprehensions are often clearer: [x**2 for x in nums] vs list(map(lambda x:x**2, nums)).

Basic map

```
nums = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x**2, nums))
print(squares)   # [1,4,9,16,25]

words = ['hello','world','python']
upper = list(map(str.upper, words))
print(upper)     # ['HELLO','WORLD','PYTHON']
```

Output:

```
[1, 4, 9, 16, 25]
['HELLO', 'WORLD', 'PYTHON']
```

Multiple iterables

```
a = [1, 2, 3]
b = [10, 20, 30]
result = list(map(lambda x,y: x+y, a, b))
print(result)    # [11, 22, 33]
```

Output:

```
[11, 22, 33]
```

Real-world: data transformation

```
raw = ['  Alice ', ' BOB', 'carol  ']
clean = list(map(str.strip, raw))  # remove spaces
clean = list(map(str.title, clean)) # title case
print(clean)   # ['Alice', 'Bob', 'Carol']

# Convert CSV row to numbers
row = '10,20,30,40,50'
nums = list(map(int, row.split(',')))
print(nums)    # [10, 20, 30, 40, 50]
```

Output:

```
['Alice', 'Bob', 'Carol']
[10, 20, 30, 40, 50]
```

max()

Syntax: max(iterable, *, key=None, default=None)

Returns the largest item. Can take an iterable or multiple arguments. key is a function applied before comparing. default is returned for empty iterables (avoids ValueError).

■ **Tip:** Use max() with key= instead of sorting a whole list just to find the largest element.

■ Basic usage

```
print(max(3, 1, 4, 1, 5, 9))     # single call
print(max([3, 1, 4, 1, 5, 9]))   # from list
print(max('z','a','m','b'))       # strings
print(max([], default='empty'))   # safe empty
```

Output:

```
9
9
z
empty
```

■ With key function

```
words = ['apple','fig','banana','kiwi']
print(max(words, key=len))           # longest word
print(min(words, key=len))           # shortest word

nums = [-5, 3, -1, 8, -9]
print(max(nums, key=abs))  # largest magnitude
```

Output:

```
banana
fig
-9
```

■ Real-world: best scoring student

```
students = [
    {'name':'Alice', 'score':85},
    {'name':'Bob',   'score':92},
    {'name':'Carol', 'score':78},
]
top = max(students, key=lambda s: s['score'])
print(f"Top student: {top['name']} ({top['score']})")
```

Output:

```
Top student: Bob (92)
```

min()

Syntax: `min(iterable, *, key=None, default=None)`

Returns the smallest item. Mirror of max(). All the same options apply: multiple args, key function, and default for empty iterables.

■ **Tip:** min() and max() with key= are often better than sorting — O(n) vs O(n log n).

■ Basic usage

```
print(min(3, 1, 4, 1, 5))         # 1
print(min([100, 50, 200, 25]))     # 25
print(min('cat','ant','bee'))      # 'ant'
print(min([], default=0))          # safe empty → 0
```

Output:

```
1
25
ant
0
```

■ With key function

```
items = [('apple',1.5),('mango',2.0),('fig',0.8)]
cheapest = min(items, key=lambda x: x[1])
print(cheapest)   # ('fig', 0.8)
```

Output:

```
('fig', 0.8)
```

■ Real-world: closest value

```
def closest(lst, target):
    return min(lst, key=lambda x: abs(x - target))

temps = [18.5, 21.3, 24.7, 19.1, 22.8]
print(closest(temps, 20))  # 19.1 — closest to 20
```

Output:

```
19.1
```

N

next()

Syntax: next(iterator, default)

Retrieves the next item from an iterator by calling its __next__() method. If the iterator is exhausted, returns default if provided, or raises StopIteration.

Tip: next() with a generator is a powerful pattern to find the first match without loading everything.

Basic usage

```
it = iter([10, 20, 30])
print(next(it))   # 10
print(next(it))   # 20
print(next(it))   # 30
print(next(it, 'done'))  # 'done' — exhausted
```

Output:

```
10
20
30
done
```

Finding first match

```
nums = [4, 7, 2, 9, 1, 6]
# First number > 5 (efficient — stops at first match)
first = next((x for x in nums if x > 5), None)
print(first)   # 7
```

Output:

```
7
```

Real-world: lazy file reading

```
# Read only until we find what we need
with open('log.txt') as f:
    error_line = next(
        (line for line in f if 'ERROR' in line),
        'No errors found'
    )
print(error_line)
```

Output:

```
[ERROR] Connection timeout
```

O

oct()

Syntax: `oct(x)`

Converts an integer to an octal (base-8) string prefixed with '0o'. The input must be an integer or an object with __index__.

■ **Tip:** Unix file permissions are octal: 0o777=rwxrwxrwx, 0o644=rw-r--r--, 0o755=rwxr-xr-x.

■ Basic conversions

```
print(oct(0))    # 0o0
print(oct(7))    # 0o7
print(oct(8))    # 0o10
print(oct(64))   # 0o100
print(oct(255))  # 0o377
print(oct(-8))   # -0o10
```

Output:

```
0o0
0o7
0o10
0o100
0o377
-0o10
```

■ File permissions (Unix)

```
perm = 0o755   # rwxr-xr-x
print(oct(perm))       # '0o755'
print(format(perm,'o'))# '755' (no prefix)
import os
# os.chmod('file.txt', 0o644)  # set permissions
```

Output:

```
0o755
755
```

open()

```
Syntax: open(file, mode='r', encoding=None, ...)
```

Opens a file and returns a file object. The mode controls read/write/binary access. Always use 'with open()' to ensure the file is closed even if an error occurs.

■ **Note:** Specify encoding='utf-8' when working with text files to avoid encoding errors on different OS.

■ **Tip:** Always use 'with open(...)' — it automatically closes the file even if an exception occurs.

■ Common modes

```
# 'r'  — read text (default)
# 'w'  — write text (overwrites)
# 'a'  — append text
# 'rb' — read binary
# 'wb' — write binary
# 'x'  — create new (fails if exists)
# 'r+' — read and write
```

■ Reading files

```
# Read entire file
with open('notes.txt', 'r', encoding='utf-8') as f:
    content = f.read()

# Read line by line (memory efficient)
with open('big.txt') as f:
    for line in f:
        print(line.strip())
```

Output:

```
line 1...
line 2...
```

■ Writing files

```
lines = ['Alice: 95', 'Bob: 87', 'Carol: 92']
with open('scores.txt', 'w') as f:
    for line in lines:
        f.write(line + '\n')

# Append to file
with open('scores.txt', 'a') as f:
    f.write('Dave: 78\n')
```

ord()

Syntax: ord(c)

Returns the Unicode code point (integer) for a single-character string. Inverse of chr(). Useful for character arithmetic, sorting by character value, and encoding.

■ **Tip:** chr() and ord() together let you do 'character math': shift letters, build ciphers, etc.

■ Basic usage

```
print(ord('A'))   # 65
print(ord('a'))   # 97
print(ord('0'))   # 48
print(ord(' '))   # 32 — space
print(ord('\n')) # 10 — newline
print(ord('♥'))   # 9829
```

Output:

```
65
97
48
32
10
9829
```

■ Character arithmetic

```
# Position of letter in alphabet
def letter_pos(c):
    return ord(c.lower()) - ord('a') + 1

print(letter_pos('A'))   # 1
print(letter_pos('Z'))   # 26
```

Output:

```
1
26
```

■ Real-world: check character type

```
def classify(c):
    code = ord(c)
    if 65 <= code <= 90:  return 'Uppercase'
    if 97 <= code <= 122: return 'Lowercase'
    if 48 <= code <= 57:  return 'Digit'
    return 'Other'

for ch in ['A','z','5','!']:
    print(f'{ch!r}: {classify(ch)}')
```

Output:

```
'A': Uppercase
'z': Lowercase
'5': Digit
'!': Other
```

P

pow()

Syntax: pow(base, exp, mod=None)

Returns base raised to the power exp. With three arguments, returns (base**exp) % mod efficiently (modular exponentiation) — crucial for cryptography.

■ **Tip:** pow(base, exp, mod) is much faster than (base**exp) % mod for large numbers — use it in crypto.

■ Basic power

```
print(pow(2, 10))    # 1024
print(pow(3, 3))     # 27
print(pow(4, 0.5))   # 2.0 — square root
print(pow(2, -1))    # 0.5 — negative exp
```

Output:

```
1024
27
2.0
0.5
```

■ Modular exponentiation

```
print(pow(2, 10, 1000))   # 1024 % 1000 = 24
print(pow(3, 200, 13))    # fast modular pow
```

Output:

```
24
9
```

■ Real-world: compound interest

```
def compound_interest(principal, rate, years):
    amount = principal * pow(1 + rate/100, years)
    return round(amount, 2)

print(compound_interest(1000, 8, 10))
print(compound_interest(5000, 5, 20))
```

Output:

```
2158.93
13266.49
```

print()

```
Syntax: print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)
```

Prints objects to a text stream (default: stdout). Objects are separated by sep and followed by end. file redirects output. flush forces the buffer to write immediately.

Tip: For progress bars/spinners use end='' and flush=True to update the same line in real time.

Basic printing

```
print('Hello, World!')          # basic
print('a', 'b', 'c')           # three args
print('a', 'b', 'c', sep='-')  # custom sep
print('No newline', end='')     # no newline
print(' <- see?')
```

Output:

```
Hello, World!
a b c
a-b-c
No newline <- see?
```

Formatting options

```
for i in range(1, 6):
    print(f'{i:2d}: {"*"*i}')
```

Output:

```
1: *
 2: **
...
5: *****
```

Write to file

```
import sys
print('This is a warning!', file=sys.stderr)

with open('output.txt','w') as f:
    for i in range(3):
        print(f'Line {i+1}', file=f)
```

Output:

```
This is a warning! (in stderr)
```

R

range()

Syntax: range(stop) / range(start, stop, step=1)

Returns an immutable sequence of integers. Memory-efficient — only stores start, stop, step, not all values. Perfect for loops, indexing, and generating number sequences.

Tip: range() is lazy — range(1_000_000) uses almost no memory. Use list(range(n)) only when needed.

Common patterns

```
print(list(range(5)))            # 0 to 4
print(list(range(1, 6)))         # 1 to 5
print(list(range(0, 10, 2)))     # even numbers
print(list(range(10, 0, -1)))    # countdown
print(list(range(0, -10, -3)))   # negative step
```

Output:

```
[0,1,2,3,4]
[1,2,3,4,5]
[0,2,4,6,8]
[10,9,8,7,6,5,4,3,2,1]
[0,-3,-6,-9]
```

Loop patterns

```
# Repeat N times
for _ in range(3):
    print('Python!')

# Index-based loop
items = ['a','b','c','d']
for i in range(len(items)-1, -1, -1):
    print(items[i], end=' ')  # reverse order
```

Output:

```
Python! (x3)
d c b a
```

Real-world: multiplication table

```
for i in range(1, 6):
    row = [f'{i*j:4d}' for j in range(1, 6)]
    print(' '.join(row))
```

Output:

```
1  2  3  4  5
2  4  6  8 10
...
```

reversed()

Syntax: reversed(sequence)

Returns a reverse iterator for a sequence. Works with any object that has __reversed__() or supports __len__() and __getitem__(). Does NOT modify the original.

Tip: For strings, slicing is often easier: 'hello'[::-1] == 'olleh'. reversed() works on sequences.

Basic usage

```
print(list(reversed([1,2,3,4,5])))  # [5,4,3,2,1]
print(list(reversed('hello')))       # ['o','l','l','e','h']
print(list(reversed(range(5))))      # [4,3,2,1,0]
print(''.join(reversed('python')))   # 'nohtyp'
```

Output:

```
[5,4,3,2,1]
['o','l','l','e','h']
[4,3,2,1,0]
nohtyp
```

Loop in reverse

```
names = ['Alice','Bob','Carol']
for name in reversed(names):
    print(name)
```

Output:

```
Carol
Bob
Alice
```

Real-world: check palindrome

```
def is_palindrome(s):
    s = s.lower().replace(' ','').replace(',','').replace('!','')
    return list(s) == list(reversed(s))

print(is_palindrome('racecar'))       # True
print(is_palindrome('A man a plan'))  # False
print(is_palindrome('madam'))         # True
```

Output:

```
True
False
True
```

round()

```
Syntax: round(number, ndigits=None)
```

Rounds a number to ndigits decimal places. With no ndigits (or None), returns an integer. Uses banker's rounding (round half to even) to reduce statistical bias.

■ **Note:** Due to floating-point representation, round(2.675, 2) may give 2.67 — use decimal module for exact money math.

■ **Tip:** Banker's rounding: round(2.5)=2 and round(3.5)=4 — rounds to the nearest EVEN number at .5.

■ **Basic rounding**

```
print(round(3.14159, 2))   # 3.14
print(round(3.14159, 4))   # 3.1416
print(round(3.5))          # 4 — banker's rounding
print(round(4.5))          # 4 — banker's rounding!
print(round(2.5))          # 2 — banker's rounding
```

Output:

```
3.14
3.1416
4
4
2
```

■ **Negative ndigits**

```
print(round(1234, -1))    # 1230 — nearest 10
print(round(1234, -2))    # 1200 — nearest 100
print(round(1234, -3))    # 1000 — nearest 1000
```

Output:

```
1230
1200
1000
```

■ **Real-world: currency & grades**

```
# Safe money rounding
price   = 19.995
tax     = price * 0.1
total   = price + tax
print(f'Total: ${round(total, 2)}')

# Grade rounding
scores  = [85.3, 72.7, 91.5, 68.4]
rounded = [round(s) for s in scores]
print(rounded)
```

Output:

```
Total: $21.99
[85, 73, 92, 68]
```

S

set()

Syntax: `set(iterable=())`

Creates a mutable, unordered collection of unique elements. Sets are great for removing duplicates, membership testing (O(1)), and mathematical set operations (union, intersection, etc.).

■ **Note:** Sets are unordered — you can't rely on the order of elements.

■ **Tip:** In tests is very fast for sets: 'x in my_set' is O(1), while 'x in my_list' is O(n).

■ Creating sets

```
s1 = set([1,2,3,2,1])   # duplicates removed
s2 = set('hello')        # unique chars
s3 = {1,2,3,4}           # literal syntax
print(s1)   # {1, 2, 3}
print(s2)   # {'h','e','l','o'}
```

Output:

```
{1, 2, 3}
{'h', 'e', 'l', 'o'}
```

■ Set operations

```
a = {1,2,3,4,5}
b = {4,5,6,7,8}
print(a & b)   # intersection: {4, 5}
print(a | b)   # union: {1,2,3,4,5,6,7,8}
print(a - b)   # difference: {1,2,3}
print(a ^ b)   # symmetric diff: {1,2,3,6,7,8}
```

Output:

```
>{4, 5}
{1,2,3,4,5,6,7,8}
{1,2,3}
{1,2,3,6,7,8}
```

■ Real-world: duplicate detection

```
emails = ['a@b.com','c@d.com','a@b.com','e@f.com']
unique = set(emails)
dupes  = [e for e in emails if emails.count(e)>1]
print('Unique:', len(unique))
print('Duplicates:', set(dupes))

# Fast membership test
banned = {'spam@evil.com','hack@bad.com'}
email  = 'spam@evil.com'
print('Blocked:', email in banned)  # O(1)
```

Output:

```
Unique: 3
Duplicates: {'a@b.com'}
Blocked: True
```

sorted()

```
Syntax: sorted(iterable, *, key=None, reverse=False)
```

Returns a new sorted list from an iterable. Original is NOT modified. key is a function applied to each element for comparison. reverse=True for descending order.

Tip: Use a tuple key like (-gpa, age) to sort by multiple fields at once without extra comparisons.

Basic sorting

```
print(sorted([3,1,4,1,5,9,2]))          # ascending
print(sorted([3,1,4,1,5], reverse=True)) # descending
print(sorted('python'))                  # alphabetical
print(sorted(['banana','apple','fig']))  # strings
```

Output:

```
[1,1,2,3,4,5,9]
[5,4,3,1,1]
['h','n','o','p','t','y']
['apple','banana','fig']
```

Sorting with key

```
words = ['banana','apple','fig','cherry']
print(sorted(words, key=len))           # by length
print(sorted(words, key=str.lower))     # case-insensitive
```

Output:

```
['fig','apple','banana','cherry']
['apple','banana','cherry','fig']
```

Real-world: sort complex data

```
students = [
    {'name':'Alice','gpa':3.8,'age':20},
    {'name':'Bob',  'gpa':3.5,'age':22},
    {'name':'Carol','gpa':3.8,'age':19},
]
# Sort by GPA desc, then age asc
result = sorted(students,
    key=lambda s: (-s['gpa'], s['age']))
for s in result:
    print(s['name'], s['gpa'], s['age'])
```

Output:

```
Carol 3.8 19
Alice 3.8 20
Bob 3.5 22
```

str()

```
Syntax: str(object='')
```

Converts any object to its string representation. Calls the object's __str__ method (or __repr__ as fallback). Also a string constructor for bytes decoding.

Tip: f-strings are usually preferred over str() for formatting: f'{value}' is cleaner and faster.

Basic conversion

```
print(str(42))          # '42'
print(str(3.14))        # '3.14'
print(str(True))        # 'True'
print(str(None))        # 'None'
print(str([1,2,3]))     # '[1, 2, 3]'
print(str({'a':1}))     # "{'a': 1}"
```

Output:

```
42
3.14
True
None
[1, 2, 3]
{'a': 1}
```

Bytes decoding

```
b = b'Hello, World!'
print(str(b, 'utf-8'))       # decode bytes
print(b.decode('utf-8'))     # same result
```

Output:

```
Hello, World!
Hello, World!
```

Real-world: building output

```
data = {'user':'Alice','score':95,'passed':True}
parts = [k + '=' + str(v) for k,v in data.items()]
print(', '.join(parts))
```

Output:

```
user=Alice, score=95, passed=True
```

sum()

Syntax: sum(iterable, start=0)

Sums all items in an iterable. The start parameter is added to the total — useful for combining sums or setting a base value. Works with any numeric types.

Tip: For strings, use ''.join(list) not sum() — sum() on strings raises TypeError.

Basic sums

```
print(sum([1,2,3,4,5]))          # 15
print(sum([1.1, 2.2, 3.3]))      # ~6.6
print(sum(range(1, 101)))        # 5050 (1+2+...+100)
print(sum([10,20,30], 100))      # 160 (start=100)
```

Output:

```
15
6.6
5050
160
```

Flatten list of lists

```
nested = [[1,2],[3,4],[5,6]]
flat = sum(nested, [])   # start=[] concatenates
print(flat)   # [1,2,3,4,5,6]
```

Output:

```
[1, 2, 3, 4, 5, 6]
```

Real-world: totaling a shopping cart

```
cart = [
    {'item':'Coffee', 'price':4.99, 'qty':2},
    {'item':'Milk',   'price':2.49, 'qty':1},
    {'item':'Bread',  'price':3.29, 'qty':1},
]
total = sum(i['price'] * i['qty'] for i in cart)
print(f'Cart total: ${total:.2f}')
```

Output:

```
Cart total: $15.76
```

super()

Syntax: super(type=None, obj_or_type=None)

Returns a proxy object that delegates method calls to the parent or sibling class. Essential for proper inheritance — lets you call the parent class's methods without naming it explicitly.

■ **Tip:** Always call super().__init__() in child classes to ensure parent initialization runs correctly.

■ **Basic inheritance**

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return f'{self.name} makes a sound'

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)   # call Animal.__init__
        self.breed = breed
    def speak(self):
        return super().speak() + ' — Woof!'

d = Dog('Rex','Labrador')
print(d.speak())
```

Output:

```
Rex makes a sound — Woof!
```

■ **Multiple inheritance (MRO)**

```
class A:
    def hello(self): return 'A'
class B(A):
    def hello(self): return 'B->' + super().hello()
class C(A):
    def hello(self): return 'C->' + super().hello()
class D(B, C):
    def hello(self): return 'D->' + super().hello()
print(D().hello())  # D->B->C->A (MRO order)
```

Output:

```
D->B->C->A
```

T

tuple()

Syntax: `tuple(iterable=())`

Creates an immutable ordered sequence. Like lists but cannot be modified after creation. Tuples are faster, use less memory, and can be used as dictionary keys.

Tip: Prefer tuples for heterogeneous data (name, age, score) and lists for homogeneous sequences.

Creating tuples

```
t1 = tuple([1,2,3])          # from list
t2 = tuple('abc')            # from string
t3 = tuple(range(5))         # from range
t4 = (1, 2, 3)               # literal syntax
t5 = (42,)                   # single-item — note comma!
print(t1, t2, t3, t5)
```

Output:

```
(1,2,3) ('a','b','c') (0,1,2,3,4) (42,)
```

Tuple unpacking

```
coords = (10, 20, 30)
x, y, z = coords      # unpack to variables
print(x, y, z)

a, *rest = (1,2,3,4,5)
print(a, rest)         # 1, [2,3,4,5]
```

Output:

```
10 20 30
1 [2, 3, 4, 5]
```

Real-world: function returning multiple values

```
def min_max_avg(nums):
    return min(nums), max(nums), sum(nums)/len(nums)

lo, hi, avg = min_max_avg([4,8,15,16,23,42])
print(f'Min:{lo} Max:{hi} Avg:{avg:.1f}')
```

Output:

```
Min:4 Max:42 Avg:18.0
```

type()

Syntax: type(object) / type(name, bases, dict)

With one argument: returns the type (class) of an object. With three arguments: dynamically creates a new class. The foundation of Python's type system.

■ **Tip:** Use isinstance() for type checking in practice — it handles inheritance and is more Pythonic.

■ Check types

```
print(type(42))          # <class 'int'>
print(type(3.14))        # <class 'float'>
print(type('hello'))     # <class 'str'>
print(type([]))          # <class 'list'>
print(type(None))        # <class 'NoneType'>
```

Output:

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'list'>
<class 'NoneType'>
```

■ type() vs isinstance()

```
class Animal: pass
class Dog(Animal): pass
d = Dog()
print(type(d) == Dog)        # True
print(type(d) == Animal)     # False — exact type
print(isinstance(d, Animal)) # True — checks parents too
```

Output:

```
True
False
True
```

■ Dynamic class creation

```
# Equivalent to: class Point: x=0; y=0
Point = type('Point', (), {'x':0,'y':0})
p = Point()
print(p.x, p.y)    # 0 0
print(type(p))     # <class '__main__.Point'>
```

Output:

```
0 0
<class '__main__.Point'>
```

V

vars()

Syntax: vars(object=None)

Returns the __dict__ attribute of an object (its namespace as a dict). Without an argument, acts like locals() and returns the current local symbol table.

■ **Tip:** vars() is great for serializing objects to dicts: json.dumps(vars(obj)) converts to JSON.

■ **Inspect an object**

```
class Config:
    host = 'localhost'
    port = 8080
    debug = True

cfg = Config()
print(vars(cfg))     # instance dict (empty)
print(vars(Config))  # class dict (has host,port,debug)
```

Output:

```
{}
mappingproxy({'host':'localhost','port':8080,...})
```

■ **Modify via vars()**

```
class Box:
    def __init__(self, w, h):
        self.w, self.h = w, h

b = Box(10, 5)
print(vars(b))           # {'w':10, 'h':5}
vars(b)['color'] = 'red' # add attribute
print(b.color)           # 'red'
```

Output:

```
{'w': 10, 'h': 5}
red
```

Z

zip()

Syntax: zip(*iterables, strict=False)

Aggregates elements from multiple iterables into tuples. Stops at the shortest iterable (by default). With strict=True, raises ValueError if iterables have different lengths.

■ **Note:** zip() stops at the SHORTEST iterable — use itertools.zip_longest() to pad with a fill value.

■ **Tip:** zip() is lazy — it only processes pairs on demand. Use list(zip(...)) to materialize all pairs.

■ **Basic zip**

```
names  = ['Alice','Bob','Carol']
scores = [85, 92, 78]
for name, score in zip(names, scores):
    print(f'{name}: {score}')
```

Output:

```
Alice: 85
Bob: 92
Carol: 78
```

■ **Unzip with ***

```
pairs = [(1,'a'),(2,'b'),(3,'c')]
nums, letters = zip(*pairs)  # unzip
print(nums)     # (1, 2, 3)
print(letters)  # ('a', 'b', 'c')
```

Output:

```
(1, 2, 3)
('a', 'b', 'c')
```

■ **Real-world: create dict from two lists**

```
keys   = ['name','age','city']
values = ['Alice', 30, 'Dhaka']
person = dict(zip(keys, values))
print(person)

# Matrix transpose
matrix = [[1,2,3],[4,5,6],[7,8,9]]
transposed = list(zip(*matrix))
for row in transposed:
    print(list(row))
```

Output:

```
{'name':'Alice','age':30,'city':'Dhaka'}
[1,4,7]
[2,5,8]
[3,6,9]
```